

day 6

إليك نوتة **Obsidian** متكاملة ومفصلة لكل سطر، وتفصيلية، وملحوظة وردت في هذا الجزء من السكتشن (Date, Functions, Scope, Hoisting) مع تفكيك التراكات البرمجية وخبايا تنفيذ المحرك (JS Engine) بالكامل:

Markdown

```
# JavaScript Core: Date Object, Functions, Scope & Hoisting
```

```
---
```

```
## 📅 1. The Date Object (كائن التاريخ والوقت)
```

الدمج لإنشاء وتعديل واستخراج التواريخ والأوقات في جافا `Date` يُستخدم كائن سكريبت.

```
### (أ) جلب التاريخ والوقت الحالي (Current Date)
```

```
```javascript
```

```
var currentDate = new Date();
```

```
console.log(currentDate); // المنطقة، الوقت الحالي بالثانية، والمنطقة (Timezone) الزمنية
```

### (ب) إنشاء تاريخ محدد (Specific Date)

لدينا طريقتان لتحديد تاريخ معين في الماضي أو المستقبل، وكل طريقة لها سلوك مختلف تماماً:

#### 1. تمرير الأرقام المفصلة (new Date(year, month, day))

⚠️ ⚠️ تركة الفهرسة الصفيرية للشهور (Zero-Indexed Months) عند إنشاء تاريخ باستخدام الأرقام، تذكر أن الشهور تبدأ من الرقم 0 وليس 1.

• شهر يناير (January) = 0

• شهر ديسمبر (December) = 11 مصفوفة الشهور في الذاكرة: [Jan, 1=Feb, 2=Mar, 3=Apr, =0]

[4=May, 5=Jun, 6=Jul, 7=Aug, 8=Sep, 9=Oct, 10=Nov, 11=Dec]

### JavaScript

```
var myNumbersDate = new Date(2024, 5, 2);
```

```
console.log(myNumbersDate); // المخرجات: Sun Jun 02 2024 (لأن رقم 5 يمثل شهر) (!يوليو وليس مايو)
```

## 2. تمرير صيغة النص new Date("month/day/year")

عند استخدام النصوص، يختلف السلوك؛ الشهور تُكتب بأرقامها الطبيعية (شهر يناير = 1، شهر يونيو = 6) والتنسيق المعتمد هو التنسيق الأمريكي المباشر.

JavaScript

```
var myStringDate = new Date("6/2/2024");
console.log(myStringDate); // المخرجات: Sun Jun 02 2024 (شهر 6 هنا هو يونيو)
(مباشرة)
```

### (ج) دوال استخراج قيم التاريخ (Date Methods)

عند قراءة تفاصيل التاريخ، تنطبق قواعد الفهرسة الصفرية (Zero-indexed) على الأيام والشهور كالتالي:

- الأيام (getDay ()): تبدأ من يوم الأحد وترمز له بالرقم 0 وحتى السبت 6. [Sunday, 1=Monday,=0]
- الشهور (getMonth ()): تبدأ من 0 وحتى 11 كما وضعنا سابقاً.

JavaScript

```
console.log(myStringDate.getDay()); // 0 => Sunday (الأحد)
console.log(myStringDate.getMonth()); // 5 => يونيو (مؤشر 5 في
المصفوفة)
console.log(myStringDate.getFullYear()); // 2024 => Number
يعيد السنة كاملة كـ
من 4 أرقام
```

## 🌸 2. Functions Deep Dive (الدوال بالتفصيل)

تنقسم الدوال في جافا سكريبت من حيث طريقة كتابتها والإعلان عنها إلى نوعين أساسيين:

### (أ) النوع الأول: Function Declaration (الإعلان الصريح)

تُكتب ببداية السطر بكلمة function متبوعة باسمها، وهي دالة مستقلة بذاتها.

JavaScript

```
function funName(){
 // implementation (التنفيذ)
}

// funName() -> calling function (استدعاء الدالة لتنفيذ ما بداخلها)

function printMessage() {
 console.log("hello");
}
```

```

}
printMessage(); // الاستدعاء الأول -> hello
printMessage(); // الاستدعاء الثاني -> hello

```

## الدوال مع المعاملات (Parameters & Arguments)

- **Parameters** : هي المتغيرات الوهمية المكتوبة في تعريف الدالة فوق (المستقبلات).
- **Arguments** : هي القيم الحقيقية التي نمررها للدالة أثناء الاستدعاء تحت لتشغيل الكود بها.

### JavaScript

```

function printMessageParam(message) {
 console.log(message);
}
printMessageParam("hello"); // Output: hello
printMessageParam("hi"); // Output: hi
printMessageParam("bye"); // Output: bye

```

## ⚠️ تراكيب ومعاملات إضافية أو ناقصة (Extra & Missing Arguments)

جافا سكريبت لا تعترض ولا تخرج خطأ (Error) إذا قمت بتمرير عدد قيم مغاير لما تتوقعه الدالة:

### JavaScript

```

function sumTwoNumbers(x, y) {
 console.log(x + y);
}

sumTwoNumbers(1, 2); // 3 (طبيعي)
sumTwoNumbers(1, 2, 5); // 3 (Extra تركة: أي قيم زائدة يتم إهمالها تماماً 🟡)
// arguments are ignored
sumTwoNumbers(1); // NaN (لأن x=1 و y فاعتبرت y و x=1 لأن NaN)
// NaN يساوي undefined + 1 و undefined

```

## 🛡️ حماية الدوال عبر كائن الفحص المدمج (arguments Object)

داخل أي دالة، يتوفر كائن مدمج خفي يشبه المصفوفة يُدعى **arguments** يحتوي على كل القيم الممررة للدالة بالترتيب. يمكن استغلاله لمنع الأخطاء وعمل حماية (Validation):

### JavaScript

```

function sumTwoNumbersValidate(x, y) {
 // التحقق من أن المستخدم مرر وسيطين بالضبط (لا أكثر ولا أقل). 1.
 if (arguments.length !== 2) {

```

```

 throw "Arguments must be 2"; // ترمي استثناء وتوقف الكود فوراً
 }

 // التحقق من أن القيم الممررة أرقام فقط وليست نصوماً 2.
 if (typeof x !== "number" || typeof y !== "number") {
 throw "Arguments must be numbers";
 }

 console.log(x + y);
}

sumTwoNumbersValidate(2, 5); // تنجح 7
// sumTwoNumbersValidate(2); // تضرب إرور Arguments must be 2
// sumTwoNumbersValidate(2, "5"); // تضرب إرور Arguments must be
// sumTwoNumbersValidate(2, "ali"); // تضرب إرور Arguments must be
// numbers

```

### تطبيق متقدم: جمع مصفوفة وسائط ديناميكية غير محددة العدد

باستخدام كائن arguments مع حلقة تكرار (for loop)، يمكننا استقبال أي عدد من الأرقام وجمعها تصاعدياً:

#### JavaScript

```

function sumAnyCount() {
 var total = 0;
 for (var i = 0; i < arguments.length; i++) {
 total += arguments[i]; // إضافة القيمة الحالية للمجموع الإجمالي
 }
 return total; // إرجاع الناتج الإجمالي وخروج من الدالة
}

console.log(sumAnyCount(5, 4, 6)); // الوسائط الممررة خلف الكواليس 15
[6, 4, 5]
console.log(sumAnyCount(5, 4, 6, 7)); // الوسائط الممررة خلف الكواليس 22
[7, 6, 4, 5]
console.log(sumAnyCount(5)); // 5
console.log(sumAnyCount()); // total لأن الحلقة لن تعمل بقيمة 0
(الابتدائية 0)

```

### ب) النوع الثاني: Function Expression (التعبير البرمجي للدالة)

هنا نقوم بإنشاء متغير عادي ونقوم بتخزين دالة مجهولة الاسم (Anonymous Function) بداخله. الدالة تُعامل كقيمة مخزنة.

#### JavaScript

```
var x = function(){
 console.log("hello");
 return 5;
};
```

في الكونسول "hello" استدعاء عادي للمتغير يطبع

```
var result = x(); // تخزين القيمة المرجوعة (5) داخل متغير جديد
console.log(result); // يطبع 5
// console.log(x()); // يطبع "hello" القيمة المرجوعة
// النهائية داخل الكونسول
```

### 3. Variables Scope (نطاق المتغيرات)

VAR vs LET vs CONST			
	var	let	const
Stored in Global Scope	✓	✗	✗
Function Scope	✓	✓	✓
Block Scope	✗	✓	✓
Can Be Reassigned?	✓	✓	✗
Can Be Redeclared?	✓	✗	✗
Can Be Hoisted?	✓	✗	✗

الـ Scope يحدد المساحة الجغرافية في الكود التي يُسمح فيها برؤية المتغير واستخدامه:

## 💡 قواعد الـ var مقابل المتغيرات الضمنية (Implicit Globally)

- المتغيرات بـ **var** : تمتلك **Global Scope** إذا عُرفت في السكريبت الخارجي، وتمتلك **Function Scope** إذا عُرفت داخل دالة (تُحبس داخل الدالة). وهي لا تحترم الـ Block scope (مثل الـ if والـ for).
- المتغيرات الضمنية ( **y = 20** ): كتابة اسم متغير وتخصيص قيمة له دون وضع **var** أو **let** أو **const**. هذا المتغير يصبح **Global** فوراً في كل السكريبت حتى لو كُتب داخل دالة أو داخل أي Block!

### 1. نطاق الـ Block المحلي وعلاقته بـ var

JavaScript

```
if (true) {
 var x = 10; // لا تحترم الـ var الـ if، تصبح
 Global
 y = 20; // implicit globally variable -> تصبح Global تلقائياً
}
console.log(x); // 10 (متشاف عادي)
console.log(y); // 20 (متشاف عادي)

for (var i = 0; i < 5; i++) {
 console.log(i); // 0 لـ 4 يطبع
}
console.log(i); // 5 (الـ i المتغير)
var (اختارقتها)
```

### 2. الحبس داخل نطاق الدالة (Function Scope)

الدالة هي النطاق الوحيد القادر على حبس متغيرات الـ var ومنع خروجها:

JavaScript

```
function cannotAccessX(){
 var x = 10; // محبوسة داخل نطاق الدالة فقط (Function scope)
}
// console.log(x); // ❌ Uncaught ReferenceError: x is not defined (فشل الوصول)
// (من الخارج)
```

## 🕵️ تحليل الألغاز الشجرية لنطاقات الدوال المتداخلة (Scope Chain Puzzles)

لغز النطاق (1): حجب المتغير الخارجي بـ **Local Var (Shadowing)**

JavaScript

```

var x = 20; // Global Variable
function func1() {
 var x = 10; // قامت بإنشاء متغير محلي جديد تماماً اسمه func1 دالة
 الخارجي
 console.log(x); // 10 (تقرأ المحلي الأقرب لها دائماً)

 function func2() {
 console.log(x); // 10 (فتجده 10 وتطبعها func1 تبحث في نطاقها الأبوي)
 }
 func2();
}
func1();
console.log(x); // 20 (الـ Global x وجود ولم يتغير بسبب وجود x الـ Global)
(داخلي مستقل)

```

## لغز النطاق (2): القراءة المباشرة من الأعلى

### JavaScript

```

var x = 20;
function func1() {
 console.log(x); // 20 (فصعدت خطوة للنطاق الأكبر، func1 محلي داخل x لم تجد)
 (Global وجلبت الـ Global)

 function func2() {
 console.log(x); // 20 (Global فلا تجد، فتصعد للـ func1 تخرج تبحث في)
 (وتجلب 20)
 }
 func2();
}
func1();
console.log(x); // 20

```

## لغز النطاق (3): ⚠️ فخ تعديل المتغير وتأثيره (Reassign)

### JavaScript

```

var x = 20; // Global
function func1() {
 var x = 10; // Global متغير محلي حبس القيمة 10 داخله وعزل الـ
 console.log(x); // 10

 function func2() {
 console.log(x); // 10 (func1 في دالة var المكتوب بـ x يقرأ)
 }
}

```

```

 }
 func2();
}
func1();
console.log(x); // 20 (الـ Global 20 يظل الخارجي)
```

ملحوظة للتفرقة لو أن كود الدالة كتب السطر هكذا: `x = 10`; بدون كلمة `var` نهائياً، لكانت هذه العملية بمثابة **(Reassign)** وإعادة تخصيص للـ **Global** الخارجي، ولأصبحت قيمته في السطر الأخير **10** بدلاً من **20**. ولكن كتابة كلمة `var` تحميه تماماً وتنشئ نسخة منفصلة للنطاق الحالي.

## 4. Hoisting (الرفع الإزاحي)

الـ **Hoisting** هو آلية داخل محرك الجافا سكريبت يقوم فيها بمسح الكود برمجياً قبل تشغيله، ويرفع تعريفات المتغيرات والدوال إلى أعلى النطاق الخاص بها (Top of scope).

### قواعد الـ Hoisting الحتمية:

1. المتغيرات المكتوبة بـ `var`: يتم رفع التعريف فقط (Declaration) إلى الأعلى وتثبيت قيمته المبدئية بـ `undefined`، أما التخصيص (Assignment) فيبقى في سطره الأصلي.
2. الـ **Function Declaration** (الدوال الصريحة): يتم رفع الدالة بكامل جسمها ومحتواها (Body) إلى الأعلى، مما يتيح استدعاءها بأمان قبل سطر تعريفها الفعلي.
3. الـ **Function Expression** (الدوال كمتغيرات): تعامل معاملة المتغيرات العادية تماماً، يُرفع الاسم فقط وتكون قيمته المبدئية `undefined`؛ وإذا استدعيتها قبل سطرها يثور المتصفح بالخطأ الشهير.

### محاكاة كواليس الـ Hoisting في الذاكرة:

#### 1. كواليس المتغيرات:

JavaScript

```

// الكود المكتوب:
console.log(x); // undefined (مرفوع بالذاكرة ومرفوع)
// undefined
var x = 10;
console.log(x); // 10 (أخذ القيمة الحقيقية)
```

الفعلي Hoisting شكل الكود في الذاكرة بعد الـ:

```

// undefined تم رفع التعريف للأعلى تلقائياً وقيمته المبدئية
// var x;
// console.log(x);
// x = 10; // التخصيص بقي مكانه في السطر الثاني
// console.log(x);
```

#### 2. كواليس الدوال الصريحة:



## JavaScript

```
printHello(); // بنجاح مذهب لأن جسم الدالة بالكامل تم رفعه قبل "hello" تطبع
التنفيذ
```

```
function printHello() {
 console.log("hello");
}
```

### ❌ 3. التركة القاتلة: استدعاء الـ Function Expression قبل سطرها

## JavaScript

```
x(); // ❌ خطأ فادح: Uncaught TypeError: x is not a function
```

```
var x = function () {
 console.log("hello");
};
```

```
// Hoisting: تفسير حدوث الخطأ خلف كواليس الـ
// undefined وقيمتها الحالية): var x; المحرك قام برفع المتغير هكذا في الأعلى
// undefined(): في السطر الأول، كأنك تقول للمحرك نفذ الآتي x() عند كتابة
// لذا يضرب الكود (Not a function)، ليس دالة قابلة للتنفيذ undefined والـ
!فوراً
```